



Security Insider Lab II

hermSec TUI Project Proposal

Group 4

1. GHAZAL POURESFANDIYAR BOROJENI
 2. SETH WHENTON
-

May 13, 2026

Contents

Abstract	iii
1 Problem Statement	1
1.1 Target Users	1
2 Proposed Solution	2
2.1 Core Idea	2
2.2 Operating Modes	2
2.2.1 Manual Mode	2
2.2.2 Automated Mode	3
3 System Workflow	4
3.1 Workflow Diagram	4
4 Tools and Technologies	5
5 Agent Design	6
5.1 Tool Based Evidence	6
5.2 Confidence Levels	6
5.3 Memory	6
5.4 Report Generation Skills	7
6 Report Format	8
6.1 Report Sections	8
6.2 Example Finding	8
7 Implementation Plan	9
7.1 MVP	9
7.2 Future Extensions	9
8 Evaluation Plan	11
8.1 Detection Test	11
8.2 Tool Comparison Test	11
8.3 Report Quality Test	11
8.4 Daily Workflow Test	11
9 Risks and Limitations	12
9.1 False Positives	12
9.2 Model Hallucination	12

Contents

9.3	Privacy	12
9.4	Scope	12
10	Conclusion	13

Abstract

hermSec TUI is a proposed CVE aware AI security review agent for software developers. The project is designed for development teams that ship code quickly and may not manually scan every change for vulnerabilities. The system will accept a GitHub repository URL or a local repository path, run several static analysis and dependency scanning tools, compare their results, and create a clear report for developers. The report will explain risky code, vulnerable dependencies, code quality concerns, severity, related CVEs or CWEs, and possible fixes.

The project will use a simple agentic AI harness. The agent will manage tool calls, decide which scanners to run, compare the results, perform its own model based review on the relevant code context, and generate a readable report. Tools such as Bandit, Semgrep, pip-audit, OSV-Scanner, and Gitleaks will provide evidence. A bring your own model approach will then be used for reasoning and explanation. This means the user can choose a model through OpenRouter, Ollama, LM Studio, or another compatible provider. The goal is to make security feedback more automatic, more understandable, and easier to include in daily development work.

1 Problem Statement

Many programmers work in teams where code is shipped fast. In such environments, developers may focus on features, bug fixes, deadlines, and deployment speed. Security checks are often delayed until later, or they are done only when someone remembers to run a scanner manually. This creates a gap between writing code and receiving security feedback.

Static security tools already exist, but their output can be difficult to understand. A scanner may produce many warnings, technical labels, and raw file references. Developers then need to decide which findings matter, which are duplicates, which are false positives, and what should be fixed first. Dependency vulnerabilities create another problem. A project can include an old package version with a known CVE, but this issue may remain unnoticed until release time or after deployment.

hermSec is designed to solve this practical problem. It aims to give developers a single assistant that can automatically review the code they shipped, compare results from different tools, check known vulnerability databases, and produce a daily report that is clear enough to act on.

1.1 Target Users

The main target users are programmers and small development teams. The project is especially useful for teams that:

- push code frequently during the day
- do not always run manual security checks
- need simple explanations rather than raw scanner output
- want to track new, open, and fixed vulnerabilities over time
- want to use their own local or cloud AI model

2 Proposed Solution

The proposed solution is **hermSec TUI**, a terminal based security review agent built around a small agentic AI harness. In the first version, a user will run the tool manually and provide either a GitHub URL or a local project folder. The agent will manage the static analysis tools, inspect the most relevant code context with model support, compare the findings, and generate a report. In a later version, the same scanning engine can be connected to the Hermes agent harness so it can run automatically at the end of the day.

2.1 Core Idea

The main idea is to combine static analysis tools, dependency vulnerability checks, agent memory, and model based review. The security tools provide evidence. The agent organizes the evidence, chooses what should be inspected further, and asks the selected model to reason over the relevant code snippets. The model helps explain findings, identify possible bugs missed by simple rules, and suggest fixes.

The model will not be treated as the only source of truth. This is important because language models can make mistakes or invent details. hermSec will only call a finding a confirmed CVE when a dependency scanner or vulnerability database provides evidence. For code patterns, the system will normally use CWE categories instead of claiming that a specific CVE exists.

2.2 Operating Modes

2.2.1 Manual Mode

Manual mode is the first version and the main demonstration version. The user runs hermSec from the terminal.

```
hermsec scan https://github.com/example/project
hermsec scan ./local-project
```

The agent scans the repository, compares scanner findings, asks the selected model to explain them, and returns a terminal report and Markdown report.

2.2.2 Automated Mode

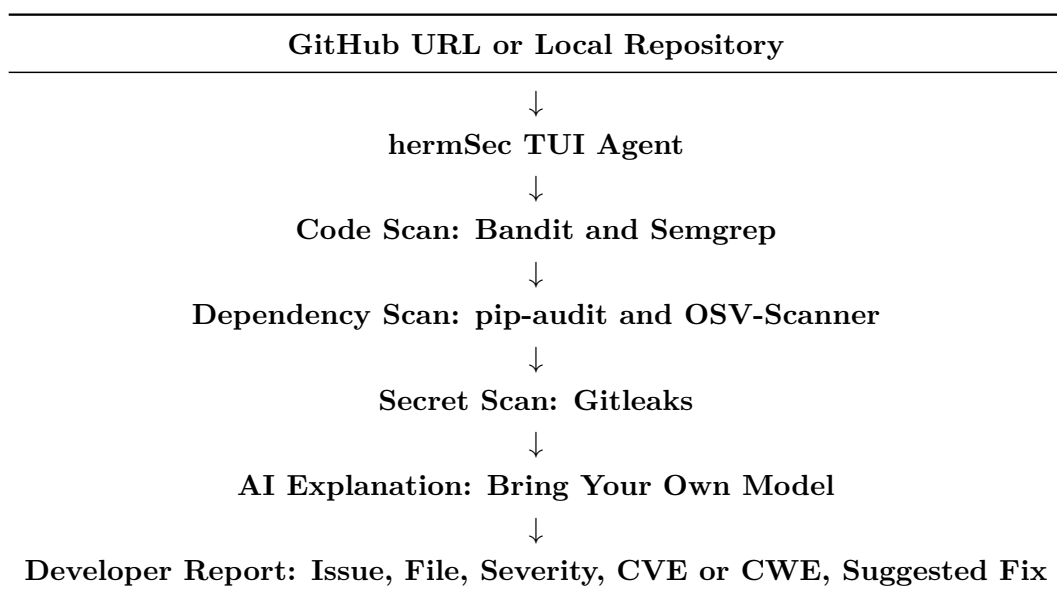
Automated mode is the future Hermes harness version. In this mode, a scheduled agent can run at the end of the day. It can review code changes, remember previous findings, and send a report to a team channel such as Telegram through an MCP integration.

3 System Workflow

The workflow of hermSec is designed to be simple and explainable.

1. The user provides a GitHub URL or local repository path.
2. The system clones or opens the repository.
3. The system detects source files and dependency files.
4. Bandit scans Python code for security issues.
5. Semgrep scans code using pattern based rules.
6. pip-audit or OSV-Scanner checks dependencies against vulnerability databases.
7. Gitleaks checks for accidentally committed secrets.
8. The agent normalizes all findings into one common format.
9. The agent compares scanner results and assigns confidence levels.
10. The selected model explains the findings using scanner evidence.
11. The system generates a report for developers.

3.1 Workflow Diagram



4 Tools and Technologies

Category	Tools and Purpose	
Programming languages	lan-	TypeScript for the terminal interface and agent flow. Python for scanner support and test repositories.
Agent layer		A small agentic AI harness for tool management, scanner orchestration, model based review, and report generation. OpenRouter Agent TUI can be used for the interactive interface. Hermes agent harness can be used later for scheduled and managed agent execution.
Model providers		OpenRouter, Ollama, LM Studio, or any OpenAI compatible model endpoint. This supports the bring your own model idea.
Static analysis		Bandit for Python security issues. Semgrep for custom code pattern detection and broader static analysis rules.
Dependency scanning		pip-audit and OSV-Scanner for package vulnerabilities and known CVE matches.
Secret scanning		Gitleaks for detecting tokens, passwords, and API keys accidentally included in code.
Vulnerability sources		OSV.dev, NVD CVE API, and GitHub Advisory Database for vulnerability information and enrichment.
Version control		Git and GitHub for repository input, commit history, and future workflow integration.
Reporting		Terminal output, Markdown report, JSON report, and optional PDF or graph based summaries.
Notifications		Telegram MCP or another messaging integration for daily developer reports.

5 Agent Design

5.1 Tool Based Evidence

hermSec will use tools first. The scanners are responsible for finding evidence. The agent then reads and compares the evidence. This makes the system more reliable than asking a model to inspect a whole repository without support.

5.2 Confidence Levels

The agent will assign confidence levels based on the source of evidence.

Confidence	Meaning
High	More than one scanner reports the same or similar issue. For example, Bandit and Semgrep both report dangerous subprocess usage.
Medium	One scanner reports a concrete issue with file and line evidence.
Low	The model reports a code quality concern without strong scanner evidence. These should be treated as suggestions, not confirmed vulnerabilities.
Confirmed CVE	A dependency name and version match a known vulnerability from pip-audit, OSV-Scanner, OSV.dev, NVD, or GitHub Advisory Database.

5.3 Memory

Memory is important because the project is not only about one scan. hermSec should remember previous findings so the daily report can show:

- new issues introduced today
- issues that remain open
- issues that were fixed

- repeated patterns that appear often in the team code
- changes in dependency risk over time

For the first implementation, memory can be stored in a local JSON file or SQLite database.

5.4 Report Generation Skills

The agent can use custom skills or report templates to produce consistent reports. These reports can include summaries, tables, severity counts, and optional graphs. The report should not only list problems. It should also explain what the programmer should do next.

6 Report Format

The report should be short enough for developers to read, but detailed enough to be useful.

6.1 Report Sections

- repository name and scan date
- scan mode, such as manual or scheduled
- tools used
- summary of critical, high, medium, and low issues
- new issues
- open issues
- fixed issues
- dependency CVEs
- code quality concerns
- suggested fixes

6.2 Example Finding

Field	Example
Issue	Possible command injection
File	scripts/run_task.py
Severity	High
Evidence	Bandit and Semgrep both reported subprocess usage with shell=True
Related weakness	CWE-78
Explanation	User controlled input may reach an operating system command.
Suggested fix	Avoid shell=True and pass command arguments as a list.
Confidence	High

7 Implementation Plan

7.1 MVP

The minimum viable version will focus on a simple agent harness for manual repository scanning. The goal is not only to run tools separately, but to let the agent manage the tools and combine their results with model based review. It will include:

- a terminal interface
- GitHub URL and local folder input
- an agentic AI SDK or harness for managing scanner tool calls
- Bandit scan
- Semgrep scan
- pip-audit or OSV-Scanner dependency scan
- Gitleaks secret scan
- normalized findings
- confidence scoring
- model based review of relevant code snippets
- bring your own model explanation
- Markdown and terminal report output

7.2 Future Extensions

After the MVP works, the project can be extended with:

- Hermes scheduled agent execution
- daily reports for code shipped that day
- Telegram MCP notifications
- trend graphs
- GitHub Actions integration

7 Implementation Plan

- pre-push hook integration
- team level dashboards

8 Evaluation Plan

The evaluation will check whether the project is useful and technically correct.

8.1 Detection Test

We will create or use a small vulnerable repository. It will include known examples such as SQL injection, command injection, hardcoded secrets, and vulnerable dependencies. hermSec should detect these issues through the selected tools.

8.2 Tool Comparison Test

We will compare the hermSec report with raw outputs from Bandit, Semgrep, pip-audit, OSV-Scanner, and Gitleaks. The goal is to confirm that hermSec does not hide important findings and that it improves readability.

8.3 Report Quality Test

We will inspect the generated reports and check whether each important finding contains:

- issue title
- file or package
- severity
- CVE or CWE when available
- scanner evidence
- explanation
- suggested fix

8.4 Daily Workflow Test

For the scheduled version, repeated scans will be used to test memory. The system should show which findings are new, which remain open, and which were fixed.

9 Risks and Limitations

9.1 False Positives

Static tools can report false positives. hermSec should not automatically claim that every warning is a real vulnerability. Confidence levels and clear evidence are needed.

9.2 Model Hallucination

The AI model may invent details if it is not guided carefully. The prompt must tell the model not to invent CVEs. A CVE should only be shown when it appears in tool evidence.

9.3 Privacy

Some repositories may be private. The system should tell users which model provider is used and what data is sent. Local models should be supported for privacy sensitive projects.

9.4 Scope

The project is defensive. It is meant to scan code and suggest fixes. It should not generate exploit code or attack live systems.

10 Conclusion

hermSec TUI proposes a practical security assistant for modern development teams. It is useful because it combines multiple scanners, CVE information, memory, model based code review, and AI explanations into one developer focused report. The first version will be a manual TUI repository scanner powered by a simple agentic harness. Later versions can run through the Hermes agent harness as an automated daily review agent. The project is related to current security trends because it connects AI assisted development, DevSecOps, vulnerability management, and software supply chain security.